Mega resumen V0.1

OCI

DFS Normal

Generalmente usaremos DFS para determinar ciclos en un grafo, además de contar el número de componentes conexas. El DFS explora el grafo en profundidad.

```
#include <iostream>
#include <vector>
using namespace std;
// Definiremos la constante NO_VISITADO como -1
#define NO_VISITADO -1

// Vector de enteros. Lo utilizaremos para representar que el nodo i
// fue visitado. 1 para visitado. -1 para NO visitado.
vector <int> visitados;
// Vector de Vectores para representar nuestra lista de adyacencia
vector <vector <int>> AdjList;
// El DFS nos permitira explorar el grafo en profundidad. Generalmente
// utilizaremos DFS para encontrar ciclos, contar componentes conexas

void dfs (int u) {
 // Marcamos el nodo u como visitado.
 visitados[u] = 1;
 // Por cada vecino que tenga el nodo u, haremos un dfs si es
 // que ese vecino no fue visitado anteriormente
 for (int j = 0; j < (int)AdjList[u].size(); j++) {
 // por comodidad, rescatamos el vecino j del nodo u
  int v = AdjList[u][j];
  // si no hemos visitado al vecino v, entonces hacemos dfs.
  // Usamos first ya que el vecino es un par (nodo,peso)
  if (visitados[v] == NO_VISITADO) {
   // dfs desde el vecino
   dfs(v);
  }
 }
}


int main() {

 // N representa la cantidad de nodos. E representa
 // la cantidad de aristas (conexiones) entre estos N nodos
 int N, E;

 // Generalmente nos daran la info de N o E.
 // Sino lo dan de manera explícita es porque se puede calcular
 cin >> N >> E;

 // Inicializamos nuestra lista de adyacencia con N
```

```
 // vectores de pares (vii) vacios
 AdjList.assign(N, vector <int>());
 // Llenamos nuestra lista de adyacencia.
 for (int i = 0; i < E; i++) {
  int u, v;
  // leemos los datos de los nodos en conexion
  cin >> u >> v;
  // Agregamos a nuestra lista la conexion entre el nodo u y el v.
  // Si los nodos parten en 1 y no 0, hacer --u, --v
  AdjList[u].push_back(v);
  // Si es que el grafo es NO dirigido, o sea, puede ir de A a B
  // y de B a A, agregamos el par inverso a la lista de adyacencia
  AdjList[v].push_back(u);
 }


 // Para sacar el numero total de componentes conexas, recorremos todos
 // nuestros nodos. Usamos para esto el vector de visitados.
 // Si el nodo i NO esta visitado, contamos una componente conexa y
 // hacemos DFS (el DFS marcara como visitados todos los nodos vecinos
 // a este)
 // Inicializamos todos nuestros nodos como NO visitados

 visitados.assign(N, NO_VISITADO);
 // Inicialmente tenemos 0 componentes conexas

 int totalComponentesConexas = 0;

 for (int i = 0; i < N; i++) {
  // si no hemos visitado este nodo
  if (visitados[i] == NO_VISITADO) {
   // contamos una componente conexa
   totalComponentesConexas++;
   // hacemos DFS desde el nodo
   dfs(i);
  }
 }

 cout << totalComponentesConexas << endl;

 return 0;
}
```

DFS Grafo Implícito (grillas, tableros)

En el caso de un grafo implícito, reemplazamos la lista de adyacencia por una matriz (que puede representar un tablero u otra cosa). Los vecinos los

generamos a partir de arreglos de cambios de coordenadas o de algún procedimiento específico determinado por el enunciado del problema

```
#include <iostream>
#include <vector>

using namespace std;

int MAX_FILAS, MAX_COLUMNAS;

// El tablero, o la grilla para leer de la entrada. Usaremos la misma grilla
// para marcar el recorrido del DFS. Si necesitamos mantener
// intacto el tablero original, entonces debemos crear
// otra matriz de visitados

vector <vector <char>> grilla;

// Arreglo de movimientos (generador de vecinos)
// Representa todos los posibles movimiento en filas y columnas.
// Es el cambio de coordenadas para cada movimiento
int mFilas[] = {1, 1, 0,-1,-1,-1, 0, 1}; // S,SE,E,NE,N,NW,W,SW
int mColumnas[] = {0, 1, 1, 1, 0,-1,-1,-1}; // S,SE,E,NE,N,NW,W,SW


bool limites (int f, int c) {
 // Verificamos limites de la grilla. Si estamos fuera, devolvemos 0
 return (f < 0 || f >= MAX_FILAS || c < 0 || c >=MAX_COLUMNAS);
}

// DFS. Recorremos la grilla buscando el caracter c1.
// Si lo encontramos lo cambiaremos por el caracter c2
// Contaremos la cantidad de cambios hechos.
// Sirve para reconocer componentes conexas en una grilla, y "pintarlas"

int dfs(int f, int c, char c1, char c2) {

 // El caracter en la posicion f, c no era el buscado.
 // Devolvemos 0 (0 caracteres c1 encontrados)
 if (grilla[f][c] != c1) {
  return 0;
 }

 // Si llegamos aca es porque estamos dentro de la grilla y
 // porque encontramos el caracter c1
 // Marcamos visitada la posicion f,c. Cambiamos c1 por c2
 grilla[f][c] = c2;

 // Hasta este punto hemos cambiado 1 caracter
 // (visto desde otra forma, nuestra componente conexa tiene 1 nodo)
 int ans = 1;
```

```
 // Tenemos en total 8 movimientos (puede variar segun el problema)
 for (int i = 0; i < 8; ++i) {
  // llamada recursiva por cada posible movimiento. Contamos cuantos
  // caraceres pintamos (o cuantos nodos tiene la componente)
  ans += dfs(f + mFilas[i], c + mColumnas[i], c1, c2);
 }

 // Devolvemos el total de elementos en la componente conexa
 // o cuantos caraceres reemplazamos, visitamos, pintamos, etc...
 return ans;
 }


int main() {

 //leemos las dimensiones
 cin >> MAX_FILAS >> MAX_COLUMNAS;

 //inicializamos la grilla con un valor por defecto
 grilla.assign(MAX_FILAS+1, vector<char> (MAX_COLUMNAS+1, ' '));

 // llenamos la matriz según el problema
 for (int i = 0; i < MAX_FILAS; i++) {
  for (int j = 0; j < MAX_COLUMNAS; j++) {
   cin >> grilla[i][j];
  }
 }


 // buscaremos cuantas componentes conexas con '1' existen.
 // Depende el ejercicio
 int totalComponentesConexas = 0;

 // recorremos la grilla
 for (int i = 0; i < MAX_FILAS; i++) {
  for (int j = 0; j < MAX_COLUMNAS; j++) {
   // si la posicion i,j no ha sido visitada
   if (grilla[i][j] == '1') {
    // contamos una componente conexa
    totalComponentesConexas++;
    // DFS. Marcamos con X los visitados. Contamos los 1 por componenetes y
    //guardamos este valor en contador. Este representa "total de nodos en la
    // componente". Se puede usar depende el problema
    int contador = dfs(i,j,'1','x');
   }
  }
 }

 cout << totalComponentesConexas;

 return 0;
}
```

BFS Normal

BFS nos permite explorar el grafo en anchura (capas). Un ejemplo para entender esto: La persona A le da un mensaje a todos sus amigos el día 1. Al día 2, cada uno de sus amigos lo hará con sus propios amigos. Así sucesivamente. Generalmente usaremos BFS encontrar caminos mínimos en grafos SIN peso. El camino mínimo puede ser, distancia, tiempo, etc...

```cpp
#include <iostream>
#include <queue>
#include <vector>
#include <climits>

using namespace std;


// Vector de Vectores para representar nuestra lista de adyacencia
vector <vector <int>> AdjList;

//Vector para almacenar la distancia / costos para visitar el nodo i
vector <int> distancias;

// Vector de int para guardar el padre/antecesor del nodo actual. Nos sirve
// para poder imprimir el camino
vector <int> p;

//Minima ruta desde inicio a destino
int inicio, destino;

// Imprimiremos el camino hasta el nodo u
void imprimirCamino (int u) {
 // si el nodo u es igual al nodo donde partimos el BFS, imprimimos el valor
 // y terminamos. inicio declarada global
 if (u == inicio) {
  cout << u;
  return;
 }

 // Llamada recursiva con el padre de u, formato de la impresion
 // inicio ->...->...-->destino
 imprimirCamino(p[u]);
```

```
 cout << "  " << u;
}


int main() {

//total de nodos (V) y total aristas (E) del grafo
 int N, E;
 cin >> N >> E;

 //asignamos tanto espacio a nuestra lista como nodos tengamos en el grafo.
 // Del tipo vector <int>

 AdjList.assign(N, vector <int> ());

 for (int i = 0; i < E; ++i) {
  int nodo1, nodo2;
  cin >> nodo1 >> nodo2;

  // Si los nodos parten de 0, lo agreamos directamente. Sino,
  // debemos restarle 1. Si el grafo es NO dirigido, tambien debemos agregar
  // el otro nodo de la relacion: AdjList[nodo2].push_back({nodo1, 0})
  // depende del ejercicio

  AdjList[nodo1].push_back(nodo2);
 }


 cin >> inicio >> destino;
 // Inicialmente, ningun nodo tiene padres asi que asignamos -1
 // para cada nodo
 p.assign(N, -1);

 // Vector para marcar el camino recorrido y guardar la distancia.
 // Inicialmente la distancia hacia cualquier nodo es INF
 // (o podria ser -1 o cualquier negativo)
 distancias.assign(N, INT_MAX);

 // La distancia al nodo inicial es 0
 distancias[inicio] = 0;
 // La cola nos permitira procesar los nodos en el orden que van siendo
 // visitados. La gran diferencia con el DFS es que aca
 // procesaremos los nodos una vez terminado de explorar
 // todos los vecinos del nodo actual, mientras que en el DFS vamos
 // visitando los vecinos inmediatamente. Ej vida diaria:
 // DFS seria estar haciendo algo y de repente me dieron ganas de hacer otra
 // cosa, entonces dejo de hacer lo que estoy haciendo
 // y empiezo a hacer esta nueva actividad. Si mientras hago esta nueva
 // actividad quiero hacer otra nueva, lo mismo. Dejo de hacer la actual y
 // empiezo la nueva. Una vez terminada la ultima actividad,
 // vuelvo atras a la anterior y la termino.
 // Voy dejando cosas en espera. BFS seria empezar la nueva actividad
```

```
 // (o todas las nuevas actividades) solo cuando termino
 // de hacer la que estoy haciendo actualmente.
 // Para generar esa espera usamos una cola(queue)
 queue <int> q;

 // procesamos el nodo actual
 q.push(inicio);

 //mientras nos queden nodos por procesar
 while (!q.empty()) {
  // Recuperamos el nodo actual. En las colas, es obligación eliminar el
  // elemento (pop) para poder pasar al siguiente. front nos devuelve el
  // nodo actual
  int actual = q.front();
  q.pop();

  // aca podemos validar si el nodo actual es el destino
  // en ese caso, hemos llegado al destino.


  //recorremos todos los vecinos del nodo actual
  for (int j = 0; j < (int)AdjList[actual].size(); j++) {
   // recuperamos la indo del vecino j
   int vecino = AdjList[actual][j];
   // sino hemos visitado al vecino j
   if (distancias[vecino] == INT_MAX) {
    distancias[vecino] = distancias[actual] + 1;
   // el padre de vecino es actual
    p[vecino] = actual;
    q.push(vecino);
   }
  }
 }

 return 0;
}
```

BFS Implícito (tablero, grillas)

Lo mismo que en el BFS anterior, pero reemplazando la lista de adyacencia por una grilla o tablero (matriz). Los vecinos los vamos a generar a partir de un arreglo de cambios de posiciones en la matriz (o cualquier procedimiento específico descrito en el enunciado del problema)

```
#include <cstdio>
```

```cpp
#include <vector>
#include <climits>
#include <queue>
#include <iostream>

using namespace std;

int MAX_FILAS, MAX_COLUMNAS;

//utilizamos un vector de vectores para representar la grilla. En este caso
//guardamos el valor de la distancia en la misma grilla
//segun el problema, puede que necesitemos crear una grilla de distancias
//adicional, para no perder la grilla de lectura
vector <vector <int>> grilla;

// arreglo de movimiento. Puede ser cualquier cosa, dependerá del problema.

int dx[] = {-2,-1, 1, 2, 2, 1,-1,-2};
int dy[] = {-1,-2,-2,-1, 1, 2, 2, 1};


bool limites (int f, int c) {
// Validamos que nos estamos moviendo dentro de los limites del tablero
  return (f >= 0 && f < MAX_FILAS && c >= 0 && c < MAX_COLUMNAS);
}


int main () {

  // par de coordenadas para el punto de partida y para el punto de destino
  // usamos par para mayor comididad

  //leemos las dimensiones de la grilla. Si no nos entregan esto,
  //debe estar defido en el enunciado del problema
  cin >> MAX_FILAS >> MAX_COLUMNAS;

  pair <int, int> partida;
  pair <int, int> destino;


  grilla.assign(MAX_FILAS, vector <int> (MAX_COLUMNAS, INT_MAX));

  // leemos datos de partida y de destino (son coordenadas x,y)
  cin >> partida.first >> partida.second;
  cin >> destino.first >> destino.second;


  // BFS
  // usamos cola de pares para representar coordenadas a procesar
  queue <pair <int, int>> q;
  q.push(partida);

  //la distancia del tablero en su punto de partida es 0
```

```
  grilla[partida.first][partida.second] = 0;

  bool encontrado = false;

  // Mientras podamos procesar nodos
  while (!q.empty()) {

    // recuperamos la coordenada actual
    pair<int, int> nodoActual = q.front();
    q.pop();

    //hemos llegado al destino
    if (nodoActual == destino) {
      encontrado = true;
      break;
    }

    // Los vecinos de la coordenadas seran todos los movimientos
    // validos desde ese punto, definidos en nuestro arreglo
    for (int i = 0; i < 8; i++) {
      int nx, ny;
      //nuevo movimiento para la coordenada x e y
      nx = nodoActual.first + dx[i];
      ny = nodoActual.second + dy[i];

      // Si estamos en una coordenada válida y
      // ademas no hemos visitado antes este nodo nx, ny

      if (limites(nx, ny) && grilla[nx][ny] == INT_MAX) {
        pair <int, int> nodoSiguiente = {nx, ny};

        // Calculamos la distancia al vecino como la distancia actual+1
        grilla[nx][ny] = grilla[nodoActual.first][nodoActual.second] + 1;

        // encolamos la siguiente coordenada
        q.push(nodoSiguiente);
      }
    }
  }



  if (encontrado) {
    //revisar como parten los nodos. Si parten en 1, se debe restar 1
    //desde un comienzo para procesar bien
    cout << grilla[destino.first][destino.second] << endl;
  } else {
    cout << -1 << endl;
  }

return 0;
}
```

Dijkstra Normal

El algoritmo de Dijsktra nos permitirá encontrar el costo mínimo entre dos nodos cualquiera del grafo. Es bastante similar al BFS, pero cambiamos el tipo de cola y cambiamos la forma de actualizar el peso (vector de distancias). Lo importante es recordar que debemos revisar si para cada nodo a procesar, existe o no algún camino previo mejor. No olvidar que en Dijkstra encolamos el par {distancia, nodo}, donde nodo es un int que representa el nodo. La lista de adyacencia mantiene el orden original

```cpp
#include <iostream>
#include <vector>
#include <queue>
#include <climits>

using namespace std;

//Lista de adyacencia. En este caso es un vector de vector de pares
//porque debemos almacenar la información del costo
vector<vector<pair<int,int>>> AdjList;

//Vector para almacenar el costo/distancia hacia el nodo i
vector <int> distancias;

int main(){

 int N, E;
 int nodoInicio, nodoDestino;

 //leemos la cantidad de nodos y aristas
 //leemos el nodo de inicio y de termino
 cin >> N >> E >> nodoInicio >> nodoDestino;
```

```
//revisamos si los nodos parten en 1 o 0. Restamos si
//parten en 1
nodoInicio--;
nodoDestino--;

//Asignamos tantos vectores de pares como nodos tenemos
AdjList.assign(N,vector<pair<int,int>>());

int nodo1, nodo2, costo;

//por cada conexion, leemos los nodos y el costo
for (int i = 0; i < E; i++){
 cin >> nodo1 >> nodo2 >> costo;
 nodo1--;
 nodo2--;

 //guardamos en la lista de adyacencia el nodo 1, el nodo 2 y el costo
 AdjList[nodo1].push_back({nodo2, costo});
 AdjList[nodo1].push_back({nodo2, costo});
}

//la distnacia para llegar a cada nodo es infinito
//revisar el maximo costo que se puede generar
//de ser necesario, cambiar a long long
distancias.assign(N, INT_MAX);

//la distancia parte en 0 desde el inicio
distancias[nodoInicio] = 0;

//la cola rara. Organizamos los nodos en la cola por su costo/distancia
//siempre procesaremos primero aquellos con menos distancia
//esta es la declaración para que la cola orden por distancias
priority_queue<pair<int,int>, vector<pair<int,int>>, greater<pair<int,int>>> pq;

//encolamos la partida. IMPORTANTE. Encolamos primero el costo
//luego el noco
pq.push({0, nodoInicio});

bool encontrado = false;

while(!pq.empty()) {

 //sacamos el par que esta al principio de la cola
 pair<int,int> f; f = pq.top();
 pq.pop();

 //organizamos la info para claridad
 //el primer valor es la distancia, el segundo el nodo actual
 int dist_actual = f.first;
 int nodo_actual = f.second;

 //si la distancia correspondiente al nodo a procesar
```

```
  //es mayor a algun valor de distancia calculado previamente
  //para ese nodo, la descartamos porque no nos sirve
  if (dist_actual> distancias[nodo_actual]){
    continue;
  }

  //si el nodo que estamos procesando es el destino, terminaoms
  if (nodo_actual == nodoDestino){
   encontrado = true;
   break;
  }

  //recorremos los vecinos del nodo actual
  for (int j = 0; j < AdjList[nodo_actual].size(); j++) {
   //guardamos el nodo vecino en una variable para comodidad
   pair<int,int> vecino = AdjList[nodo_actual][j];

   //si la distancia al nodo actual mas el tramo para llegar al vecino
   //es menor que alguna distancia previamente calculada hacia el vecino
   //quiere decir que tenemos un camino menor, por lo tanto
   //actualizamos la distancia

   //aca, el vecino tiene la info del costo para llegar a el en el second
   //dado que asi lo agregamos a la lista de adyaencia
   if (distancias[nodo_actual] + vecino.second < distancias[vecino.first]){
    distancias[vecino.first] = distancias[nodo_actual] + vecino.second;

    //encolamos al reves, para respetar el orden de la cola, que ordena
    //por distancias/costo
    pq.push({distancias[vecino.first], vecino.first});
   }
  }
 }

 //imprimos la distancia
 if (encontrado) {
  cout <<distancias[nodoDestino] << endl;
 }



return 0;
}
```

Dijkstra Implícito (grillas, tableros)

Lo mismo que el ejemplo anterior, sin embargo, reemplazamos la lista de adyacencia por una matriz (tablero, grilla). La estrategia es bastante similar, pero utilizamos una matriz para representar los costos. No olvidar que en Dijkstra encolamos el par {distancia, nodo}, donde nodo es un par de coordenadas en un grafo implícito.

```
#include <iostream>
#include <vector>
#include <queue>
#include <climits>

using namespace std;

int MAX_FILAS, MAX_COLUMNAS;

long long INF = LLONG_MAX;

//posiciones de los vecinos en la grilla. Depende del problema
int dr[] = { -1, 0, 1, 0};
int dc[] = { 0, 1, 0, -1};


bool limites (int x, int y) {
 return (x >= 0 && x < MAX_FILAS && y >= 0 && y < MAX_COLUMNAS);
}


//esto dependera 100% del ejercicio
//en este caso considero un tablero que
vector <vector <long long> > grilla;

//para guardar el costo/distancia para llegar al nodo x,y
//para este ejemplo, consideraremos costos muy grandes
//por lo que trabajaremos con long long. Recordar que los nodos ahora
//son coordenadas
vector <vector <long long> > distancias;


int main() {

 // total de nodos y aristas/conexiones. Aca esto
```

```
// representan coordenadas donde cada valor i,j de la grilla
// representa el costo de ese nodo

int N, M;
cin >> N >> M;

grilla.assign(N, vector<long long>(M, 0));



//leo la grilla
for (int i = 0; i < N; i++) {
 for (int j = 0; j < M; j++) {
  cin >>grilla[i][j];
  }
 }

//las distancias por defecto son inalcanzables para cada nodo
distancias.assign(N, vector <long long> (M, INF));


// por comodida, creo pares para representar la coordenada x,y
pair <int, int> partida;
pair <int, int> destino;

partida.first = 0;
partida.second = 0;

destino.first = N - 1;
destino.second = M - 1;

distancias[partida.first][partida.second] = grilla[partida.first][partida.second];


// la cola rarita. Esta vez representamos el par
// {costo, {x, y}}. Un par de pares, con el first como el costo y
// el second.first como la coordenada x y el second.second como la coordenada y

priority_queue <pair <long long, pair <int, int> >, vector<pair <long long, pair <int,
int> >>, greater<pair <long long, pair <int, int> >> > pq;

//en colamos el par (costo, {x, y})
pq.push({distancias[0][0], {0,0}});

bool encontrado = false;

while (!pq.empty()) {

 //recuperamos el par al frente de la cola
 pair <long long, pair <int, int> > f = pq.top();
 pq.pop();

 //recuperamos la info por comodidad
```

```
  long long distancia = f.first;
  pair <int, int> nodo_actual = f.second;

  //si ya llegamos al destino, terminamos
  if (destino == nodo_actual) {
    encontrado = true;
    break;
  }

  //recorremos los vecinos. En este caso, como es una grilla
  //determinado por los arreglos de posiciones en la grilla
  for (int j = 0; j < 4; j++) {
   int nx = nodo_actual.first + dr[j];
   int ny = nodo_actual.second + dc[j];

   if (limites(nx, ny)) {
    //si la distancia al nodo actual mas el salto al vecino
    //es mejor que algo ya calculado, actualizamos
    if (distancia + grilla[nx][ny] < distancias[nx][ny]) {
     distancias[nx][ny] = distancia + grilla[nx][ny];
     //encolamos al reves
     pq.push({distancias[nx][ny], {nx, ny}});
     }
   }
  }
 }

 cout << distancias[destino.first][destino.second] << endl;

return 0;

}
```

En caso de que las ciudades fuesen string, podemos usar un map para darle valores numéricos a cada ciudad, de manera que podamos implementar los algoritmos como siempre. Solo debemos crear un map que guarde cada ciudad única con un valor numérico (que será el nodo), y de ahí en adelante, usar ese valor int. Ejemplo:

```
#include <iostream>
#include <map>
#include <vector>
```

```cpp
using namespace std;

vector <vector <int>> AdjList;


int main () {


 int indice = 0;
 int N, E;
 string ciudad1, ciudad2;
 cin >> N >> E;


 AdjList.assign(N, vector <int>());

 map <string, int> stringToIndice;

 for (int i = 0; i < E; i++) {

       cin >> ciudad1 >> ciudad2;

       // le damos un valor numero a cada ciudad diferente
        if (stringToIndice.count(ciudad1) == 0) {
              stringToIndice[ciudad1] = indice;
              indice++;
       }

       // le damos un valor numero a cada ciudad diferente
       if (stringToIndice.count(ciudad2) == 0) {
              stringToIndice[ciudad2] = indice;
              indice++;
       }

       //agreamos los nodos a la lista de adyacencia
       //con su correspondiente valor int
       AdjList[stringToIndice[ciudad1]].push_back(stringToIndice[ciudad2]);
       AdjList[stringToIndice[ciudad2]].push_back(stringToIndice[ciudad1]);


       //de ahora en adelante todo igual que siempre

       }


 return 0;

 }



 return 0;
```

```
}
```

Si por casualidad nos piden imprimir un camino con los nombres de las ciudades, una posible estrategia sería tener dos maps: uno de string a int para darle un valor númerico a cada ciudad, y luego, un map de int a string, para saber a qué a qué número corresponde cada índice. De esta manera podremos imprimir algo como intToString[0] y debería imprimir el string a la cual se le fue asignado un 0.

Otros códigos de utilidad

- **Buscar todas las ocurrencias de un substring dentro de un string**

.find() nos devuelve la posición en donde se encuentra la primera ocurrencia del string. Si queremos buscar la siguiente debemos volver a llamar .find(), pero esta vez, un caracter después desde la primera ocurrencia. Si quiero buscar todos los caracteres, repetir hasta que la respuesta de **.find** sea **string::npos.** Ver ejemplo:

```
//resto del codigo

string texto;
getline(cin, texto);
```

```
string nombre = "Texto a buscar";

int found = texto.find(nombre);
int cont = 0;
while (found != string::npos) {
     cont++;
     cout << found << endl;
     found = texto.find(nombre, found+1);

     if (cont == N) {
     break;
     }
}
```

Importante: Si tengo que leer un número y luego un string que incluya espacios, es decir, usando getline(cin, mi_string), no olvidar utilizar luego de leer el número getchar() o cin.ignore() para que pueda leer bien. Ver ejemplo

```
//resto del codigo
int N;
string texto;

cin >> N;
cin.ignore();  // o usar un getchar()

getline(cin, texto);
```

- **Partir un string por espacios (u otro tipo de delimitador, incluso, varios a la vez):**

Podemos hacer esto de varias formas. Una de estas es a través del comando strtok (incluir usando **#include <cstring>**). Esta función de C nos permite partir un string por "token" o "delimitadores" los que se almacenarán en un nuevo string. Luego, podemos hacer lo que queramos con este string (imprimirlo mediante cout, guardarlo en un vector, un map, etc). Vamos a pasar el string a la función tal y como se muestra en el código, para que de esta forma se pueda procesar.

```
//resto del codigo
string texto = "Este texto tiene varias palabras, ademas, contiene espacios y
comas!!!!"

/* aca removemos los espacios. Pasamos el string a la funcion como
&nombre_del_string[0] */
```

```
char* token = strtok(&texto[0], " ");

//Esta vez guardaremos en un vector de strings

vector <string> palabras;

while (token != NULL) {
    palabras.push_back(token);
    token = strtok(NULL, " "); //siguiente palabra. Parte en NULL
}

cout << palabras.size() << endl;

//imprimimos todas las palabras del vector
for (int i = 0; i < palabras.size(); i++) {
       cout << "La palabra " << i << " es: " << palabras[i] << " y tiene un tamano
de " << palabras[i].size() << endl;

 }
```

En este ejemplo, como estamos partiendo el string por espacios “ “, token tendrá estos valores: “Este” “texto” “tiene” “varias” “palabras,” “además,” “contiene” “espacios” “y” “comas!!!!”. **Importante:** como no se están limpiando las comas, estas forman parte del string. Lo mismo que los signos de explicación. Si queremos quitarlos, debemos agregarlos al token como se ve a continuación:

```
//resto del codigo
string texto = "Este texto tiene varias palabras, ademas, contiene espacios y
comas!!!!"

/* aca removemos los espacios. Pasamos el string a la funcion como
&nombre_del_string[0] */

//incluismo ahora en el token el espacio, las comas y los signos
char* token = strtok(&texto[0], " ,!");

//misma idea, pero esta vez usaremos map para contar frecuencias

map <string, int> frecuencias;

while (token != NULL) {
    frecuencias[token]++;
    token = strtok(NULL, " ,!"); //siguiente palabra. Parte en NULL
}

//imprimimos todas las palabras del map
```

```
for (auto it = frecuencias.begin(); it != frecuencias.end(); i++) {
       cout << "La palabra " << (*it).first << " aparece un total de: " <<
(*it).second << " veces " << endl;

 }
```

Ahora, ya no se verán ni las comas, los espacios ni las exclamaciones en cada uno de los string.

Otros códigos varios (DP + Strings)

- **Distancia de edición:** cuando queremos obtener el costo mínimo para pasar de una palabra a otra ya sea agregando una letra, quitando una letra o reemplazando una letra por otra. Ejemplo, pasar de "Matemática" a "Informática". **IMPORTANTE**: En este ejercicio, el costo por realizar alguna de estas operaciones es de 1, y el objetivo es obtener el mínimo número de cambios. No confundir este ejercicio con otros, donde por ejemplo, te digan cual es el costo mínimo de pasar una palabra a otra, pero te agregan un diccionario de palabras o te indican que hay relaciones entre una palabra y otra. En este caso se puede resolver con GRAFO Y BFS.

  Código:

```
#include <bits/stdc++.h>

using namespace std;

int min(int x, int y, int z) {
 return min(min(x, y), z);
}

int main() {
```

```
 string str1, str2;
 cin >> str1 >> str2;

 int m = str1.size();
 int n = str2.size();

 int dp[m + 1][n + 1];

 for (int i = 0; i <= m; i++) {
   for (int j = 0; j <= n; j++) {
     if (i == 0) {
       dp[i][j] = j;
     } else if (j == 0) {
       dp[i][j] = i;
     } else if (str1[i - 1] == str2[j - 1]) {
       dp[i][j] = dp[i - 1][j - 1];
     } else {
       dp[i][j] = 1 + min(dp[i][j - 1], // Insertar
                          dp[i - 1][j], // Remover
                          dp[i - 1][j - 1]); // Reemplazar
       }
    }
 }

 cout << dp[m][n] << endl;

return 0;

}
```

- **Substring común más grande:**

En este caso, queremos encontrar el largo del substring común más grande entre dos cadenas de textos. Por ejemplo, str1 = “OldSite:GeeksforGeeks.org”, str2 = “NewSite:GeeksQuiz.com”. En este caso la respuesta es 10 y corresponde al substring Geeks.

```
//Implementación de GeeksForGeeks

#include <bits/stdc++.h>
using namespace std;

int main() {

  string str1 = "OldSite:GeeksforGeeks.org";
  string str2 = "NewSite:GeeksQuiz.com";
```

```
  int m = str1.length();
  int n = str2.length();


  // Creamos la tabla de DP

  int dp[2][m + 1];
  int res = 0;

  dp[0][0] = 0;
  dp[1][0] = 0;

  for (int i = 1; i <= n; i++) {
       for (int j = 1; j <= m; j++) {
       if (str1[i - 1] == str2[j - 1]) {
          dp[i % 2][j] = dp[(i - 1) % 2][j - 1] + 1;
          if (dp[i % 2][j] > res) {
             res = dp[i % 2][j];
          }
       }
       else {
          dp[i % 2][j] = 0;
       }
    }
  }

  cout << res << endl;


  return 0;
}
```

El siguiente código, en vez de imprimir el tamaño del substring más largo, se devuelve cuál es el substring, en caso que queramos conocer cual es la cadena.

```
#include <bits/stdc++.h>

using namespace std;

int main() {

 //o leer desde el teclado
 string str1 = "GeeksforGeeksJAJAJAJA";
 string str2 = "GeJAJAJAJAeksQuiz";

 int m = str1.length();
 int n = str2.length();

 // Guardamos el tamano del substring comun mas largo
```

```
 int result = 0;

 // Variable para guardar el final del substring comun mas largo en X
 int end;

 // Matriz para guardar el resultado de dos filas consecutivas
 int len[2][n + 1];

 // Variable para representar la fila actual de la matriz
 int currRow = 0;

 // Iterar sobre todos los valores de i y j
 // largo[currRow][j] almacena el largo del substring comun mas largo
 // en el rango X[0..i] y Y[0..j]]

 for (int i = 0; i <= m; i++) {
   for (int j = 0; j <= n; j++) {
       if (i == 0 || j == 0) {
       len[currRow][j] = 0;
    }
    else if (str1[i - 1] == str2[j - 1]) {
       len[currRow][j] = len[1 - currRow][j - 1] + 1;
       if (len[currRow][j] > result) {
         result = len[currRow][j];
         end = i - 1;
       }
    }
    else {
       len[currRow][j] = 0;
    }
   }

   // hacemos que la fila actual sea la fila anterior
   currRow = 1 - currRow;
 }

 // Si no hay ningun substring comun, imprimimos -1
 if (result == 0) {
   cout << "-1" << endl;
 }

 // Longest common substring is from index
 // end - result + 1 to index end in X.
 // El substring comun mas largo es desde indice end - result + 1 en str1
 string substring = str1.substr(end - result + 1, result);

 cout << substring << endl;

 return 0;
}
```

- **Secuencia común más larga**

A diferencia del substring común más largo, la subsecuencia común no necesariamente tiene que ser con caracteres contínuos. En el siguiente ejemplo, Input: S1 = “AGGTAB”, S2 = “GXTXAYB”, la secuencia común más grande tiene un tamaño de 4, y corresponde a GTAB. En este otro ejemplo: Input: S1 = “BD”, S2 = “ABCD”, la subsecuencia común más larga tiene tamaño 2 (BD).

```
//geeokforgeeks
#include <bits/stdc++.h>
using namespace std;

int main() {

 string str1 = "AGGTAB";
 string str2 = "GXTXAYB";
 int m = str1.size();
 int n = str2.size();

 // Inicializamos una matriz de tamaño (m+1)*(n+1)
 int L[m + 1][n + 1];


 // El siguiente paso construyen L[m+1][n+1] al estilo "bottom up".
 // La posición L[i][j] contiene el tamaño de la subsecuencia
 // comun mas larga de tamano X[0..i-1] y Y[0..j-1]]

 for (int i = 0; i <= m; i++) {
   for (int j = 0; j <= n; j++) {
     if (i == 0 || j == 0) {
       L[i][j] = 0;
     } else if (str1[i - 1] == str2[j - 1]) {
       L[i][j] = L[i - 1][j - 1] + 1;
      } else {
       L[i][j] = max(L[i - 1][j], L[i][j - 1]);
      }
    }
 }

 cout << L[m][n] << endl;

 return 0;
}
```

#include <bits/stdc++.h>

```
using namespace std;

#define repf(i,k,n) for(int i=k; i<n; i++)
#define rep(i,n) for(int i=0; i<n; i++)

typedef vector<int> vi;
typedef long long ll;



int main(){
        string s;
        cin >> s;
        sort(s.begin(),s.end());
        vector<string> salida;
        salida.push_back(s);
   while (next_permutation(s.begin(), s.end())) {
      salida.push_back(s);
   }
   cout << salida.size() << endl;
   rep(i,salida.size()){
        cout << salida[i]<< endl;
   }
   return 0;
}
```